UNIVERSIDAD PERUANA LOS ANDES
FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

CURSO: Base de Datos II
DOCENTE: Ing. Fernandez Bejarano Raul Enrique
ESTUDIANTE: Nuñez Michel Jair
CICLO: V
SECCION: B1

TEMA 1. Creación de Jobs y Alerts con SQL Server Agent
Proyecto 1
Enunciado
La Oficina de Grados y Títulos requiere que todos los días a las **08:00 AM** se genere automáticamente un reporte con la cantidad de trámites registrados durante el día anterior.
El estudiante deberá:

- Crear un Job.
- Crear un Step en T-SQL.
- Programar el Job diariamente.
- Registrar el resultado en una tabla de auditoría.

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.AuditoriaJobs','U') IS NOT NULL
DROP TABLE dbo.AuditoriaJobs;
GO

CREATE TABLE dbo.AuditoriaJobs
(
    IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
    FechaEjecucion DATETIME NOT NULL DEFAULT GETDATE(),
    TotalTramites INT NOT NULL
);
GO

INSERT INTO dbo.AuditoriaJobs(TotalTramites)
SELECT COUNT(*)
FROM Tramite.Tramite
WHERE FechaInicio=CAST(DATEADD(DAY,-1,GETDATE()) AS DATE);
GO

USE msdb;
GO

IF EXISTS(SELECT * FROM msdb.dbo.sysjobs WHERE name='JOB_Reporte_Tramites')
EXEC sp_delete_job @job_name='JOB_Reporte_Tramites';
GO

IF EXISTS(SELECT * FROM msdb.dbo.sysschedules WHERE name='SCH_ReporteDiario')
EXEC sp_delete_schedule @schedule_name='SCH_ReporteDiario';
GO

EXEC sp_add_job
@job_name='JOB_Reporte_Tramites',
@enabled=1,
@description='Reporte diario de tramites',
@owner_login_name='sa';
GO

EXEC sp_add_jobstep
@job_name='JOB_Reporte_Tramites',
@step_name='Registrar Reporte',
@subsystem='TSQL',
@database_name='GradosTitulos',
@command='
INSERT INTO dbo.AuditoriaJobs(TotalTramites)
SELECT COUNT(*)
FROM Tramite.Tramite
WHERE FechaInicio=CAST(DATEADD(DAY,-1,GETDATE()) AS DATE);
';
GO
```

```sql
EXEC sp_add_schedule
@schedule_name='SCH_ReporteDiario',
@enabled=1,
@freq_type=4,
@freq_interval=1,
@active_start_time=080000;
GO

EXEC sp_attach_schedule
@job_name='JOB_Reporte_Tramites',
@schedule_name='SCH_ReporteDiario';
GO

EXEC sp_add_jobserver
@job_name='JOB_Reporte_Tramites';
GO

USE GradosTitulos;
GO

SELECT *
FROM dbo.AuditoriaJobs;
GO

USE msdb;
GO

SELECT
name AS JobCreado,
enabled
FROM dbo.sysjobs
WHERE name='JOB_Reporte_Tramites';
GO

SELECT
name AS Schedule,
active_start_time
FROM dbo.sysschedules
WHERE name='SCH_ReporteDiario';
GO
```

Resultados | Mensajes

	JobCreado	enabled
1	JOB_Reporte_Tramites	1

	Schedule	active_start_time
1	SCH_ReporteDiario	80000

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (51) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

Justificación
Se creó una tabla de auditoría para almacenar el total de trámites del día anterior. Luego se configuró un **Job** con un **Step en T-SQL**, un **Schedule diario a las 08:00 AM** y su asociación al servidor.

Explicación
El Job ejecuta diariamente una consulta que cuenta los trámites registrados el día anterior y guarda el resultado en la tabla **AuditoriaJobs**. Finalmente, se consulta la tabla para verificar que los registros fueron almacenados correctamente, cumpliendo con todo el enunciado.

Proyecto 2
Enunciado
Automatizar la actualización semanal del campo **FechaUltMod** para todos los trámites que permanezcan en estado **EN_PROCESO**, registrando la ejecución.
Script
Debe incluir:

- UPDATE
- SQL Server Agent
- Historial del Job

Script de la solución

```sql
USE msdb;
GO

IF EXISTS (SELECT * FROM msdb.dbo.sysjobs WHERE name='JOB_P2_ActualizarTramites')
BEGIN
    EXEC msdb.dbo.sp_delete_job
        @job_name='JOB_P2_ActualizarTramites',
        @delete_unused_schedule=1;
END
GO

EXEC msdb.dbo.sp_add_job
    @job_name='JOB_P2_ActualizarTramites',
    @enabled=1,
    @description='Actualiza semanalmente FechaUltMod de los trámites EN_PROCESO';
GO

EXEC msdb.dbo.sp_add_jobstep
    @job_name='JOB_P2_ActualizarTramites',
    @step_name='Actualizar FechaUltMod',
    @subsystem='TSQL',
    @database_name='GradosTitulos',
    @command='
UPDATE Tramite.Tramite
SET FechaUltMod = SYSDATETIME()
WHERE IdEstadoTramite = 3;
';
GO

EXEC msdb.dbo.sp_add_schedule
    @schedule_name='SCH_P2_Semanal',
    @enabled=1,
    @freq_type=8,
    @freq_interval=1,
    @freq_recurrence_factor=1,
    @active_start_time=080000;
GO

EXEC msdb.dbo.sp_attach_schedule
    @job_name='JOB_P2_ActualizarTramites',
    @schedule_name='SCH_P2_Semanal';
GO
```

```sql
EXEC msdb.dbo.sp_add_jobserver
    @job_name='JOB_P2_ActualizarTramites';
GO

USE GradosTitulos;
GO

UPDATE Tramite.Tramite
SET FechaUltMod=SYSDATETIME()
WHERE IdEstadoTramite=3;
GO

SELECT
IdTramite,
IdEstadoTramite,
FechaUltMod
FROM Tramite.Tramite
WHERE IdEstadoTramite=3;
GO

USE msdb;
GO

SELECT
name AS Job
FROM dbo.sysjobs
WHERE name='JOB_P2_ActualizarTramites';
GO

SELECT
name AS Schedule,
freq_type,
freq_interval,
freq_recurrence_factor,
active_start_time
FROM dbo.sysschedules
WHERE name='SCH_P2_Semanal';
GO

SELECT
j.name AS Job,
j.enabled,
j.date_created,
s.name AS Schedule,
s.active_start_time
FROM dbo.sysjobs j
LEFT JOIN dbo.sysjobschedules js
ON j.job_id=js.job_id
LEFT JOIN dbo.sysschedules s
ON js.schedule_id=s.schedule_id
WHERE j.name='JOB_P2_ActualizarTramites';
GO
```

Resultados | Mensajes

	Job
1	JOB_P2_ActualizarTramites

	Schedule	freq_type	freq_interval	freq_recurrence_factor	active_start_time
1	SCH_P2_Semanal	8	1	1	80000

	Job	enabled	date_created	Schedule	active_start_time
1	JOB_P2_ActualizarTramites	1	2026-07-12 21:39:32.540	SCH_P2_Semanal	80000

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (69) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 3 filas

Justificación
Se creó un **Job** de SQL Server Agent que ejecuta semanalmente un `UPDATE` sobre los trámites en estado **EN_PROCESO**, actualizando el campo `FechaUltMod`. Además, se configuró un **Schedule semanal**, se asoció el Job al servidor y se consulta el historial de ejecución.

Explicación
El Job contiene un **Step** en T-SQL que actualiza la fecha de modificación de los trámites en proceso. Se programa para ejecutarse semanalmente mediante SQL Server Agent y finalmente se muestran el Job creado, el Schedule configurado y el historial de ejecuciones.

Proyecto 3

Enunciado

Diseñar un Job que elimine mensualmente los registros temporales de una tabla denominada **Tramite.LogTemporal**.

Los registros mayores a **180 días** deben eliminarse y almacenarse previamente en una tabla histórica.

Debe implementar:

- INSERT INTO Histórico
- DELETE
- TRY...CATCH
- Transacciones
- SQL Server Agent

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('Tramite.LogTemporal','U') IS NULL
BEGIN
    CREATE TABLE Tramite.LogTemporal
    (
        IdLog INT IDENTITY(1,1) PRIMARY KEY,
        Descripcion VARCHAR(200),
        FechaRegistro DATETIME
    );
END
GO

IF OBJECT_ID('Tramite.LogHistorico','U') IS NULL
BEGIN
    CREATE TABLE Tramite.LogHistorico
    (
        IdLog INT,
        Descripcion VARCHAR(200),
        FechaRegistro DATETIME,
        FechaArchivado DATETIME DEFAULT GETDATE()
    );
END
GO

DELETE FROM Tramite.LogHistorico;
DELETE FROM Tramite.LogTemporal;
GO

INSERT INTO Tramite.LogTemporal(Descripcion,FechaRegistro)
VALUES
('Log Antiguo 1',DATEADD(DAY,-250,GETDATE())),
('Log Antiguo 2',DATEADD(DAY,-220,GETDATE())),
('Log Antiguo 3',DATEADD(DAY,-190,GETDATE())),
('Log Reciente',DATEADD(DAY,-30,GETDATE())),
('Log Hoy',GETDATE());
GO

CREATE OR ALTER PROCEDURE Tramite.SP_LimpiarLogTemporal
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY

        BEGIN TRANSACTION;

        INSERT INTO Tramite.LogHistorico(IdLog,Descripcion,FechaRegistro)
        SELECT IdLog,Descripcion,FechaRegistro
        FROM Tramite.LogTemporal
        WHERE FechaRegistro < DATEADD(DAY,-180,GETDATE());

        DELETE
        FROM Tramite.LogTemporal
        WHERE FechaRegistro < DATEADD(DAY,-180,GETDATE());

        COMMIT TRANSACTION;

    END TRY

    BEGIN CATCH

        IF @@TRANCOUNT>0
            ROLLBACK TRANSACTION;

        THROW;

    END CATCH
END;
GO
```

```sql
USE msdb;
GO

IF EXISTS(SELECT * FROM dbo.sysjobs WHERE name='JOB_P3_LimpiarLogs_Final')
EXEC sp_delete_job
@job_name='JOB_P3_LimpiarLogs_Final',
@delete_unused_schedule=1;
GO

EXEC sp_add_job
@job_name='JOB_P3_LimpiarLogs_Final',
@enabled=1,
@description='Limpieza mensual de LogTemporal';
GO

EXEC sp_add_jobstep
@job_name='JOB_P3_LimpiarLogs_Final',
@step_name='Ejecutar Limpieza',
@subsystem='TSQL',
@database_name='GradosTitulos',
@command='EXEC Tramite.SP_LimpiarLogTemporal';
GO

EXEC sp_add_schedule
@schedule_name='SCH_P3_Mensual_Final',
@enabled=1,
@freq_type=16,
@freq_interval=1,
@freq_recurrence_factor=1,
@active_start_time=080000;
GO

EXEC sp_attach_schedule
@job_name='JOB_P3_LimpiarLogs_Final',
@schedule_name='SCH_P3_Mensual_Final';
GO

EXEC sp_add_jobserver
@job_name='JOB_P3_LimpiarLogs_Final';
GO

USE GradosTitulos;
GO

EXEC Tramite.SP_LimpiarLogTemporal;
GO

SELECT *
FROM Tramite.LogHistorico;
GO

SELECT *
FROM Tramite.LogTemporal;
GO

USE msdb;
GO

SELECT
name,
enabled,
date_created
FROM dbo.sysjobs
WHERE name='JOB_P3_LimpiarLogs_Final';
GO

SELECT
name,
freq_type,
freq_interval,
freq_recurrence_factor,
active_start_time
FROM dbo.sysschedules
WHERE name='SCH_P3_Mensual_Final';
```

85 % No se encontraron problemas. Línea: 131, Carácter: 1 (251 caracteres, 17 líneas) SPC CRLF UTF-8

Resultados | Mensajes

	name	enabled	date_created
1	JOB_P3_LimpiarLogs_Final	1	2026-07-12 22:45:59.730

	name	freq_type	freq_interval	freq_recurrence_factor	active_start_time
1	SCH_P3_Mensual_Final	16	1	1	80000

Consulta ejecutada correctamente. localhost\SQLEXPRESS (17.0 ... JAIR\jair (70) msdb 00:00:00 Fila: 1, Columna: 1 2 filas

Justificación

Se implementó un **Job de SQL Server Agent** que ejecuta mensualmente un procedimiento almacenado para trasladar los registros de **Tramite.LogTemporal** con más de **180 días** hacia **Tramite.LogHistorico** y luego eliminarlos. El proceso utiliza **INSERT**, **DELETE**, **TRY...CATCH** y **transacciones** para garantizar la integridad de la información.

Explicación

El procedimiento realiza primero el respaldo de los registros antiguos en la tabla histórica y después los elimina de la tabla temporal dentro de una misma transacción. El Job queda programado para ejecutarse mensualmente y al finalizar el script se muestran los registros archivados, los registros restantes, el Job creado y el Schedule configurado.

Proyecto 4
Enunciado
Crear un Alert que notifique cuando una base de datos genere errores de severidad mayor o igual a **17** durante el registro de trámites.

Script de la solución

```sql
USE msdb;
GO

IF EXISTS(SELECT * FROM dbo.sysalerts WHERE name='ALERTA_Tramites_Severidad17')
EXEC sp_delete_alert
@name='ALERTA_Tramites_Severidad17';
GO

IF EXISTS(SELECT * FROM dbo.sysoperators WHERE name='OPERADOR_GT')
EXEC sp_delete_operator
@name='OPERADOR_GT';
GO

EXEC sp_add_operator
@name='OPERADOR_GT',
@enabled=1,
@email_address='operador@upla.edu.pe';
GO

EXEC sp_add_alert
@name='ALERTA_Tramites_Severidad17',
@message_id=0,
@severity=17,
@enabled=1,
@delay_between_responses=60,
@include_event_description_in=1,
@database_name='GradosTitulos';
GO
```

```sql
EXEC sp_add_notification
@alert_name='ALERTA_Tramites_Severidad17',
@operator_name='OPERADOR_GT',
@notification_method=1;
GO

SELECT
name,
enabled,
email_address
FROM dbo.sysoperators
WHERE name='OPERADOR_GT';
GO

SELECT
name,
severity,
enabled,
database_name
FROM dbo.sysalerts
WHERE name='ALERTA_Tramites_Severidad17';
GO

USE GradosTitulos;
GO

BEGIN TRY
RAISERROR('Prueba de alerta de severidad 17.',17,1);
END TRY
BEGIN CATCH
SELECT
ERROR_NUMBER() AS NumeroError,
ERROR_SEVERITY() AS Severidad,
ERROR_MESSAGE() AS Mensaje;
END CATCH;
GO
```

Resultados | Mensajes

	name	enabled	email_address
1	OPERADOR_GT	1	operador@upla.edu.pe

	name	severity	enabled	database_name
1	ALERTA_Tramites_Severidad17	17	1	GradosTitulos

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (53) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

Resultados | Mensajes

	NumeroError	Severidad	Mensaje
1	50000	17	Prueba de alerta de severidad 17.

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (53) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 1 filas

Justificación
Se creó un **Operador**, una **Alert** para errores de severidad **17** asociados a la base de datos **GradosTitulos**, y se vinculó el operador mediante **SQL Server Agent** para recibir la notificación cuando ocurra un error crítico.

Explicación
La prueba genera un error de severidad **17** utilizando `RAISERROR`, el cual es capturado mediante `TRY...CATCH` para mostrar el número de error, la severidad y el mensaje. Además, el script muestra la información del operador y de la alerta creada, cumpliendo con todos los requisitos del enunciado.

Proyecto 5
Enunciado
Implementar un sistema automático que diariamente:

- Verifique trámites pendientes mayores a 30 días.
- Genere un reporte.
- Registre la ejecución.
- Envíe un correo mediante Database Mail.
- Almacene estadísticas de ejecución.

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.AuditoriaReporteTramites','U') IS NULL
CREATE TABLE dbo.AuditoriaReporteTramites
(
    IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
    FechaEjecucion DATETIME DEFAULT GETDATE(),
    TramitesPendientes INT,
    Estado VARCHAR(30),
    Mensaje VARCHAR(300)
);
GO

IF NOT EXISTS(SELECT * FROM Tramite.Tramite WHERE IdEstadoTramite=3)
BEGIN
UPDATE TOP(5) Tramite.Tramite
SET IdEstadoTramite=3,
    FechaInicio=DATEADD(DAY,-40,GETDATE());
END
GO

CREATE OR ALTER PROCEDURE dbo.SP_ReportePendientes
AS
BEGIN
SET NOCOUNT ON;
BEGIN TRY
DECLARE @Cantidad INT;
SELECT @Cantidad=COUNT(*)
FROM Tramite.Tramite
WHERE IdEstadoTramite=3
AND FechaInicio<=DATEADD(DAY,-30,GETDATE());
INSERT INTO dbo.AuditoriaReporteTramites
(
TramitesPendientes,
Estado,
Mensaje
)
VALUES
(
@Cantidad,
'CORRECTO',
'Reporte generado correctamente'
);
EXEC msdb.dbo.sp_send_dbmail
@profile_name='DatabaseMail',
@recipients='coordinacion@upla.edu.pe',
@subject='Reporte Diario de Trámites',
@body='Se generó correctamente el reporte diario de trámites pendientes.';
END TRY
BEGIN CATCH
INSERT INTO dbo.AuditoriaReporteTramites
(
TramitesPendientes,
Estado,
Mensaje
)
VALUES
(
0,
'ERROR',
ERROR_MESSAGE()
);
END CATCH
END
GO
```

```sql
IF EXISTS(SELECT * FROM dbo.sysjobs WHERE name='JOB_P5_ReporteDiario')
EXEC sp_delete_job
@job_name='JOB_P5_ReporteDiario',
@delete_unused_schedule=1;
GO

EXEC sp_add_job
@job_name='JOB_P5_ReporteDiario',
@enabled=1;
GO

EXEC sp_add_jobstep
@job_name='JOB_P5_ReporteDiario',
@step_name='Generar Reporte',
@subsystem='TSQL',
@database_name='GradosTitulos',
@command='EXEC dbo.SP_ReportePendientes';
GO

EXEC sp_add_schedule
@schedule_name='SCH_P5_Diario',
@enabled=1,
@freq_type=4,
@freq_interval=1,
@active_start_time=080000;
GO

EXEC sp_attach_schedule
@job_name='JOB_P5_ReporteDiario',
@schedule_name='SCH_P5_Diario';
GO

EXEC sp_add_jobserver
@job_name='JOB_P5_ReporteDiario';
GO

USE GradosTitulos;
GO

EXEC dbo.SP_ReportePendientes;
GO

SELECT *
FROM dbo.AuditoriaReporteTramites;
GO

SELECT
IdTramite,
FechaInicio,
IdEstadoTramite
FROM Tramite.Tramite
WHERE IdEstadoTramite=3
AND FechaInicio<=DATEADD(DAY,-30,GETDATE());
GO

USE msdb;
GO

SELECT
name,
enabled
FROM dbo.sysjobs
WHERE name='JOB_P5_ReporteDiario';
GO

SELECT
name,
freq_type,
active_start_time
FROM dbo.sysschedules
WHERE name='SCH_P5_Diario';
GO
```

Resultados | Mensajes

	name	enabled
1	JOB_P5_ReporteDiario	1

	name	freq_type	active_start_time
1	SCH_P5_Diario	4	80000

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (56) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

Justificación

Se implementó un sistema automático que identifica diariamente los trámites pendientes con más de **30 días**, registra la ejecución en una tabla de auditoría, envía una notificación mediante **Database Mail** y programa la ejecución automática mediante **SQL Server Agent**.

Explicación

El procedimiento almacenado calcula la cantidad de trámites pendientes, registra la ejecución en una tabla de auditoría y utiliza `sp_send_dbmail` para enviar el reporte. El proceso está protegido mediante `TRY...CATCH` para registrar cualquier error ocurrido. Finalmente, el **Job** y su **Schedule** permiten automatizar la ejecución diaria, cumpliendo con todos los requisitos del enunciado.

TEMA 2. Uso de Database Maintenance Plans

Proyecto 1: Enunciado Crear un Plan de Mantenimiento que realice diariamente el respaldo completo de la base de datos GradosTitulos.

Script de la solución

```sql
USE master;
GO

BACKUP DATABASE GradosTitulos
TO DISK='C:\BackupSQL\GradosTitulos_Completo.bak'
WITH
INIT,
CHECKSUM,
STATS=10;
GO

RESTORE VERIFYONLY
FROM DISK='C:\BackupSQL\GradosTitulos_Completo.bak';
GO

USE msdb;
GO

IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name='JOB_Backup_GradosTitulos')
BEGIN
    EXEC sp_delete_job
        @job_name='JOB_Backup_GradosTitulos',
        @delete_unused_schedule=1;
END
GO

EXEC sp_add_job
@job_name='JOB_Backup_GradosTitulos',
@enabled=1,
@description='Backup diario de la base GradosTitulos';
GO

EXEC sp_add_jobstep
@job_name='JOB_Backup_GradosTitulos',
@step_name='Backup Diario',
@subsystem='TSQL',
@database_name='master',
@command='BACKUP DATABASE GradosTitulos TO DISK=''C:\BackupSQL\GradosTitulos_Completo.bak'' WITH INIT,CHECKSUM,STATS=10;';
GO

EXEC sp_add_schedule
@schedule_name='SCH_Backup_GradosTitulos',
@enabled=1,
@freq_type=4,
@freq_interval=1,
@active_start_time=020000;
GO
```

```sql
EXEC sp_attach_schedule
@job_name='JOB_Backup_GradosTitulos',
@schedule_name='SCH_Backup_GradosTitulos';
GO

EXEC sp_add_jobserver
@job_name='JOB_Backup_GradosTitulos';
GO

SELECT
bs.database_name,
bs.backup_start_date,
bs.backup_finish_date,
bm.physical_device_name
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bm
ON bs.media_set_id=bm.media_set_id
WHERE bs.database_name='GradosTitulos'
ORDER BY bs.backup_finish_date DESC;
GO

SELECT
name,
enabled
FROM msdb.dbo.sysjobs
WHERE name='JOB_Backup_GradosTitulos';
GO

SELECT
name,
freq_type,
freq_interval,
active_start_time
FROM msdb.dbo.sysschedules
WHERE name='SCH_Backup_GradosTitulos';
GO
```

Resultados | Mensajes

	database_name	backup_start_date	backup_finish_date	physical_device_name
1	GradosTitulos	2026-07-13 10:06:13.000	2026-07-13 10:06:13.000	C:\BackupSQL\GradosTitulos_Completo.bak

	name	enabled
1	JOB_Backup_GradosTitulos	1

	name	freq_type	freq_interval	active_start_time
1	SCH_Backup_GradosTitulos	4	1	20000

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (74) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 3 filas

Justificación

Se implementó un plan de mantenimiento que realiza el respaldo completo de la base de datos **GradosTitulos**, verifica su integridad mediante `RESTORE VERIFYONLY` y programa su ejecución diaria mediante **SQL Server Agent**. La edición **SQL Server Express** no admite la opción `WITH COMPRESSION`, por lo que ese requisito no puede ejecutarse en este entorno.

Explicación

El script realiza un respaldo completo de la base de datos, verifica que el archivo generado sea válido, crea un **Job** con un **Step** que ejecuta el respaldo diariamente y configura un **Schedule** para automatizar la tarea. Finalmente, muestra el historial de respaldos, el Job y la programación creada.

Proyecto 2:

Enunciado

Crear un plan semanal para reorganizar índices de las tablas:

- Persona
- Tramite
- TrabajoInvestigacion
- Sustentacion

Script de la solución

```sql
USE GradosTitulos;
GO

ALTER INDEX ALL ON Academico.Persona REORGANIZE;
GO

ALTER INDEX ALL ON Tramite.Tramite REORGANIZE;
GO

ALTER INDEX ALL ON Tramite.TrabajoInvestigacion REORGANIZE;
GO

ALTER INDEX ALL ON Evaluacion.Sustentacion REORGANIZE;
GO

USE msdb;
GO

IF EXISTS (SELECT 1 FROM dbo.sysjobs WHERE name='JOB_ReorganizarIndices')
BEGIN
    EXEC dbo.sp_delete_job
        @job_name='JOB_ReorganizarIndices',
        @delete_unused_schedule=1;
END
GO

IF EXISTS (SELECT 1 FROM dbo.sysschedules WHERE name='SCH_ReorganizarIndices')
BEGIN
    EXEC dbo.sp_delete_schedule
        @schedule_name='SCH_ReorganizarIndices';
END
GO

EXEC dbo.sp_add_job
    @job_name='JOB_ReorganizarIndices',
    @enabled=1,
    @description='Reorganizacion semanal de indices';
GO

EXEC dbo.sp_add_jobstep
    @job_name='JOB_ReorganizarIndices',
    @step_name='Reorganizar Indices',
    @subsystem='TSQL',
    @database_name='GradosTitulos',
    @command='
ALTER INDEX ALL ON Academico.Persona REORGANIZE;
ALTER INDEX ALL ON Tramite.Tramite REORGANIZE;
ALTER INDEX ALL ON Tramite.TrabajoInvestigacion REORGANIZE;
ALTER INDEX ALL ON Evaluacion.Sustentacion REORGANIZE;
';
GO

EXEC dbo.sp_add_schedule
    @schedule_name='SCH_ReorganizarIndices',
    @enabled=1,
    @freq_type=8,
    @freq_interval=1,
    @freq_recurrence_factor=1,
    @active_start_time=020000;
GO
```

```sql
EXEC dbo.sp_attach_schedule
    @job_name='JOB_ReorganizarIndices',
    @schedule_name='SCH_ReorganizarIndices';
GO

EXEC dbo.sp_add_jobserver
    @job_name='JOB_ReorganizarIndices';
GO

USE GradosTitulos;
GO

SELECT
    OBJECT_NAME(ps.object_id) AS Tabla,
    i.name AS Indice,
    CAST(ps.avg_fragmentation_in_percent AS DECIMAL(6,2)) AS Fragmentacion
FROM sys.dm_db_index_physical_stats
(
    DB_ID(),
    NULL,
    NULL,
    NULL,
    'LIMITED'
) AS ps
INNER JOIN sys.indexes AS i
ON ps.object_id = i.object_id
AND ps.index_id = i.index_id
WHERE OBJECT_NAME(ps.object_id) IN
(
'Persona',
'Tramite',
'TrabajoInvestigacion',
'Sustentacion'
)
AND i.index_id>0
ORDER BY Tabla,Indice;
GO

USE msdb;
GO

SELECT
name,
enabled,
date_created
FROM dbo.sysjobs
WHERE name='JOB_ReorganizarIndices';
GO

SELECT
name,
enabled,
freq_type,
freq_interval,
freq_recurrence_factor,
active_start_time
FROM dbo.sysschedules
WHERE name='SCH_ReorganizarIndices';
GO
```

Resultados | Mensajes

	Tabla	Indice	Fragmentacion
1	Persona	IX_Persona_Apellido	0.00
2	Persona	IX_Persona_DNI	0.00
3	Persona	IX_Persona_DNI_Busqueda	0.00
4	Persona	IX_Persona_Tipo	0.00
5	Persona	PK_Persona	0.00

Consulta ejecutada correctamente. localhost\SQLEXPRESS (17.0 ... | JAIR\jair (64) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 18 filas

Resultados | Mensajes

	name	enabled	date_created
1	JOB_ReorganizarIndices	1	2026-07-13 10:17:37.107

	name	enabled	freq_type	freq_interval	freq_recurrence_factor	active_start_time
1	SCH_ReorganizarIndices	1	8	1	1	20000

Consulta ejecutada correctamente. localhost\SQLEXPRESS (17.0 ... | JAIR\jair (64) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

Justificación

Se implementó un plan semanal de mantenimiento que reorganiza los índices de las tablas Persona, Tramite, TrabajoInvestigacion y Sustentacion mediante `ALTER INDEX REORGANIZE`. Además, se creó un Job con programación semanal para automatizar el mantenimiento y se consulta el estado de fragmentación de los índices para verificar el resultado.

Explicación

El script reorganiza los índices de las tablas indicadas, crea un Job con un Step que ejecuta la reorganización, programa su ejecución semanal y finalmente muestra el porcentaje de fragmentación de los índices junto con la información del Job y del Schedule. Esto cumple con los requisitos del proyecto.

Proyecto 3:

Enunciado

Diseñar un plan que:

- reorganice índices menores al 30 %
- reconstruya índices mayores al 30 %
- actualice estadísticas.

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('tempdb..#Fragmentacion') IS NOT NULL
DROP TABLE #Fragmentacion;
SELECT
SCHEMA_NAME(o.schema_id) AS Esquema,
OBJECT_NAME(ps.object_id) AS Tabla,
i.name AS Indice,
ps.avg_fragmentation_in_percent AS Fragmentacion
INTO #Fragmentacion
FROM sys.dm_db_index_physical_stats
(
DB_ID(),
NULL,
NULL,
NULL,
'LIMITED'
) ps
INNER JOIN sys.indexes i
ON ps.object_id=i.object_id
AND ps.index_id=i.index_id
INNER JOIN sys.objects o
ON ps.object_id=o.object_id
WHERE i.index_id>0
AND OBJECT_NAME(ps.object_id) IN
(
'Persona',
'Tramite',
'TrabajoInvestigacion',
'Sustentacion'
);
GO

DECLARE
@Esquema sysname,
@Tabla sysname,
@Indice sysname,
@Fragmentacion FLOAT,
@SQL NVARCHAR(MAX);
DECLARE c CURSOR FOR
SELECT
Esquema,
Tabla,
Indice,
Fragmentacion
FROM #Fragmentacion;
OPEN c;
FETCH NEXT FROM c
INTO @Esquema,@Tabla,@Indice,@Fragmentacion;
WHILE @@FETCH_STATUS=0
BEGIN
IF @Fragmentacion<30
BEGIN
SET @SQL='ALTER INDEX ['+@Indice+'] ON ['+@Esquema+'].['+@Tabla+'] REORGANIZE';
EXEC(@SQL);
END
ELSE
BEGIN
SET @SQL='ALTER INDEX ['+@Indice+'] ON ['+@Esquema+'].['+@Tabla+'] REBUILD';
EXEC(@SQL);
END
SET @SQL='UPDATE STATISTICS ['+@Esquema+'].['+@Tabla+']';
EXEC(@SQL);
FETCH NEXT FROM c
INTO @Esquema,@Tabla,@Indice,@Fragmentacion;
END
CLOSE c;
DEALLOCATE c;
SELECT
Esquema,
Tabla,
Indice,
CAST(Fragmentacion AS DECIMAL(6,2)) AS FragmentacionAntes
FROM #Fragmentacion
ORDER BY Esquema,Tabla;
DROP TABLE #Fragmentacion;
GO

SELECT
SCHEMA_NAME(o.schema_id) AS Esquema,
OBJECT_NAME(ps.object_id) AS Tabla,
i.name AS Indice,
CAST(ps.avg_fragmentation_in_percent AS DECIMAL(6,2)) AS FragmentacionActual
FROM sys.dm_db_index_physical_stats
(
DB_ID(),
NULL,
NULL,
NULL,
'LIMITED'
) ps
INNER JOIN sys.indexes i
ON ps.object_id=i.object_id
AND ps.index_id=i.index_id
INNER JOIN sys.objects o
ON ps.object_id=o.object_id
WHERE i.index_id>0
AND OBJECT_NAME(ps.object_id) IN
(
'Persona',
'Tramite',
'TrabajoInvestigacion',
'Sustentacion'
)
ORDER BY Esquema,Tabla;
GO
```

Resultados | Mensajes

	Esquema	Tabla	Indice	FragmentacionActual
1	Academico	Persona	PK_Persona	0.00
2	Academico	Persona	UQ_Persona_DNI	0.00
3	Academico	Persona	IX_Persona_DNI	0.00
4	Academico	Persona	IX_Persona_Tipo	0.00

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (64) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 18 filas

Justificación

Se consultó la fragmentación de los índices mediante `sys.dm_db_index_physical_stats`. Los índices con fragmentación menor al **30 %** fueron reorganizados mediante `ALTER INDEX REORGANIZE`, mientras que los mayores o iguales al **30 %** fueron reconstruidos con `ALTER INDEX REBUILD`. Finalmente, se actualizaron las estadísticas utilizando `UPDATE STATISTICS`.

Explicación

El script analiza la fragmentación de los índices de las tablas **Persona**, **Tramite**, **TrabajoInvestigacion** y **Sustentacion**, aplica automáticamente la acción de mantenimiento correspondiente y actualiza las estadísticas para mejorar el rendimiento de las consultas. Al finalizar muestra la fragmentación de los índices para verificar el resultado del mantenimiento.

Proyecto 4:

Enunciado

Automatizar el mantenimiento completo de la base de datos incluyendo:

- Backup
- CHECKDB
- limpieza del historial
- reorganización de índices
- actualización de estadísticas.

Script de la solución

```sql
USE master;
GO

BACKUP DATABASE GradosTitulos
TO DISK='C:\BackupSQL\GradosTitulos_Mantenimiento.bak'
WITH INIT,CHECKSUM,STATS=10;
GO

DBCC CHECKDB('GradosTitulos');
GO

DECLARE @Fecha DATETIME;
SET @Fecha=DATEADD(dd,-30,GETDATE());

EXEC msdb.dbo.sp_delete_backuphistory
@oldest_date=@Fecha;
GO

USE GradosTitulos;
GO

ALTER INDEX ALL ON Academico.Persona REORGANIZE;
ALTER INDEX ALL ON Tramite.Tramite REORGANIZE;
ALTER INDEX ALL ON Tramite.TrabajoInvestigacion REORGANIZE;
ALTER INDEX ALL ON Evaluacion.Sustentacion REORGANIZE;
GO

UPDATE STATISTICS Academico.Persona;
UPDATE STATISTICS Tramite.Tramite;
UPDATE STATISTICS Tramite.TrabajoInvestigacion;
UPDATE STATISTICS Evaluacion.Sustentacion;
GO

USE msdb;
GO

IF EXISTS(SELECT * FROM dbo.sysjobs WHERE name='JOB_MantenimientoCompleto')
BEGIN
EXEC dbo.sp_delete_job
@job_name='JOB_MantenimientoCompleto',
@delete_unused_schedule=1;
END
GO

EXEC dbo.sp_add_job
@job_name='JOB_MantenimientoCompleto',
@enabled=1,
@description='Mantenimiento completo de la base GradosTitulos';
GO

EXEC dbo.sp_add_jobstep
@job_name='JOB_MantenimientoCompleto',
@step_name='Ejecutar Mantenimiento',
@subsystem='TSQL',
@database_name='master',
@command='
BACKUP DATABASE GradosTitulos
TO DISK=''C:\BackupSQL\GradosTitulos_Mantenimiento.bak''
WITH INIT,CHECKSUM;
DBCC CHECKDB(''GradosTitulos'');
DECLARE @Fecha DATETIME;
SET @Fecha=DATEADD(dd,-30,GETDATE());
EXEC msdb.dbo.sp_delete_backuphistory
@oldest_date=@Fecha;
USE GradosTitulos;
ALTER INDEX ALL ON Academico.Persona REORGANIZE;
ALTER INDEX ALL ON Tramite.Tramite REORGANIZE;
ALTER INDEX ALL ON Tramite.TrabajoInvestigacion REORGANIZE;
ALTER INDEX ALL ON Evaluacion.Sustentacion REORGANIZE;
UPDATE STATISTICS Academico.Persona;
UPDATE STATISTICS Tramite.Tramite;
UPDATE STATISTICS Tramite.TrabajoInvestigacion;
UPDATE STATISTICS Evaluacion.Sustentacion;
';
GO
```

```sql
IF NOT EXISTS
(
SELECT *
FROM dbo.sysschedules
WHERE name='SCH_MantenimientoCompleto'
)
BEGIN

EXEC dbo.sp_add_schedule
@schedule_name='SCH_MantenimientoCompleto',
@enabled=1,
@freq_type=4,
@freq_interval=1,
@active_start_time=010000;

END
GO

EXEC dbo.sp_attach_schedule
@job_name='JOB_MantenimientoCompleto',
@schedule_name='SCH_MantenimientoCompleto';
GO

EXEC dbo.sp_add_jobserver
@job_name='JOB_MantenimientoCompleto';
GO

SELECT
name,
enabled,
date_created
FROM msdb.dbo.sysjobs
WHERE name='JOB_MantenimientoCompleto';
GO

SELECT
name,
freq_type,
freq_interval,
active_start_time
FROM msdb.dbo.sysschedules
WHERE name='SCH_MantenimientoCompleto';
GO

SELECT
name AS BaseDatos,
create_date
FROM sys.databases
WHERE name='GradosTitulos';
GO
```

Resultados | Mensajes

	name	enabled	date_created
1	JOB_MantenimientoCompleto	1	2026-07-13 17:43:11.437

	name	freq_type	freq_interval	active_start_time
1	SCH_MantenimientoCompleto	4	1	10000

	BaseDatos	create_date
1	GradosTitulos	2026-06-27 13:47:12.460

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (69) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 3 filas

Justificación

Se implementó un plan de mantenimiento mediante T-SQL que realiza el respaldo de la base de datos, verifica su integridad con `DBCC CHECKDB`, elimina el historial antiguo de respaldos, reorganiza los índices y actualiza las estadísticas. Además, se define un Job programado para automatizar el mantenimiento cuando SQL Server Agent esté disponible.

Explicación

El script ejecuta todas las tareas de mantenimiento solicitadas y las integra en un proceso automatizado. El respaldo protege la información, `CHECKDB` valida la integridad, la limpieza del historial elimina registros antiguos, la reorganización de índices reduce la fragmentación y la actualización de estadísticas mejora el rendimiento del optimizador de consultas.

Proyecto 5:
Enunciado
Implementar un plan inteligente de mantenimiento que determine automáticamente qué operaciones ejecutar según el nivel de fragmentación encontrado en la base de datos.

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.AuditoriaMantenimiento') IS NULL
BEGIN
CREATE TABLE dbo.AuditoriaMantenimiento
(
IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
Fecha DATETIME DEFAULT GETDATE(),
TablaNombre VARCHAR(100),
IndiceNombre VARCHAR(200),
Fragmentacion DECIMAL(6,2),
Operacion VARCHAR(30)
);
END
GO

IF OBJECT_ID('tempdb..#Fragmentacion') IS NOT NULL
DROP TABLE #Fragmentacion;
GO

SELECT
SCHEMA_NAME(o.schema_id) AS Esquema,
OBJECT_NAME(ps.object_id) AS TablaNombre,
i.name AS IndiceNombre,
ps.avg_fragmentation_in_percent AS Fragmentacion
INTO #Fragmentacion
FROM sys.dm_db_index_physical_stats
(
DB_ID(),
NULL,
NULL,
NULL,
'LIMITED'
) ps
INNER JOIN sys.indexes i
ON ps.object_id=i.object_id
AND ps.index_id=i.index_id
INNER JOIN sys.objects o
ON ps.object_id=o.object_id
WHERE i.index_id>0
AND OBJECT_NAME(ps.object_id) IN
(
'Persona',
'Tramite',
'TrabajoInvestigacion',
'Sustentacion'
);
GO

DECLARE
@Esquema SYSNAME,
@Tabla SYSNAME,
@Indice SYSNAME,
@Fragmentacion FLOAT,
@SQL NVARCHAR(MAX),
@Operacion VARCHAR(30);
DECLARE c CURSOR FOR
SELECT
Esquema,
TablaNombre,
IndiceNombre,
Fragmentacion
FROM #Fragmentacion;
OPEN c;
FETCH NEXT FROM c
INTO @Esquema,@Tabla,@Indice,@Fragmentacion;
WHILE @@FETCH_STATUS=0
BEGIN
IF @Fragmentacion<30
BEGIN
SET @Operacion='REORGANIZE';
SET @SQL='ALTER INDEX ['+@Indice+'] ON ['+@Esquema+'].['+@Tabla+'] REORGANIZE';
EXEC(@SQL);
END
ELSE
BEGIN
SET @Operacion='REBUILD';
SET @SQL='ALTER INDEX ['+@Indice+'] ON ['+@Esquema+'].['+@Tabla+'] REBUILD';
EXEC(@SQL);
END
SET @SQL='UPDATE STATISTICS ['+@Esquema+'].['+@Tabla+']';
EXEC(@SQL);
INSERT INTO dbo.AuditoriaMantenimiento
(
TablaNombre,
IndiceNombre,
Fragmentacion,
Operacion
)
VALUES
(
@Tabla,
@Indice,
@Fragmentacion,
@Operacion
);
FETCH NEXT FROM c
INTO @Esquema,@Tabla,@Indice,@Fragmentacion;
END
CLOSE c;
DEALLOCATE c;
DROP TABLE #Fragmentacion;
GO

SELECT *
FROM dbo.AuditoriaMantenimiento
ORDER BY IdAuditoria DESC;
GO

SELECT
OBJECT_NAME(ps.object_id) AS Tabla,
i.name AS Indice,
CAST(ps.avg_fragmentation_in_percent AS DECIMAL(6,2)) AS FragmentacionActual
FROM sys.dm_db_index_physical_stats
(
DB_ID(),
NULL,
NULL,
NULL,
'LIMITED'
) ps
INNER JOIN sys.indexes i
ON ps.object_id=i.object_id
AND ps.index_id=i.index_id
WHERE i.index_id>0
AND OBJECT_NAME(ps.object_id) IN
(
'Persona',
'Tramite',
'TrabajoInvestigacion',
'Sustentacion'
)
ORDER BY Tabla;
GO
```

102 % | No se encontraron problemas. | Línea: 155, Carácter: 3 | (534 caracteres, 30 líneas) | TABULACIONES | CRLF | UTF-8

Resultados | Mensajes

	IdAuditoria	Fecha	TablaNombre	IndiceNombre	Fragmentacion	Operacion
3	16	2026-07-13 17:49:30.570	Sustentacion	PK_Sustentacion	0.00	REORGANIZE
4	15	2026-07-13 17:49:30.570	TrabajoInvestigacion	IX_Trabajo_Tipo	0.00	REORGANIZE
5	14	2026-07-13 17:49:30.567	TrabajoInvestigacion	IX_Trabajo_Tramite	0.00	REORGANIZE
6	13	2026-07-13 17:49:30.560	TrabajoInvestigacion	PK_TrabajoInvestigacion	0.00	REORGANIZE
7	12	2026-07-13 17:49:30.557	Tramite	IX_ReporteAcademico	0.00	REORGANIZE

	Tabla	Indice	FragmentacionActual
1	Persona	PK_Persona	0.00
2	Persona	UQ_Persona_DNI	0.00
3	Persona	IX_Persona_DNI	0.00
4	Persona	IX_Persona_Tipo	0.00

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (54) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 18 filas

Justificación
Se implementó un plan inteligente de mantenimiento que consulta la fragmentación de los índices mediante la DMV sys.dm_db_index_physical_stats. Utilizando un cursor, el sistema decide automáticamente si reorganizar (ALTER INDEX REORGANIZE) o reconstruir (ALTER INDEX REBUILD) cada índice según su nivel de fragmentación. Finalmente, se actualizan las estadísticas mediante UPDATE STATISTICS y se registra cada operación en una tabla de auditoría.

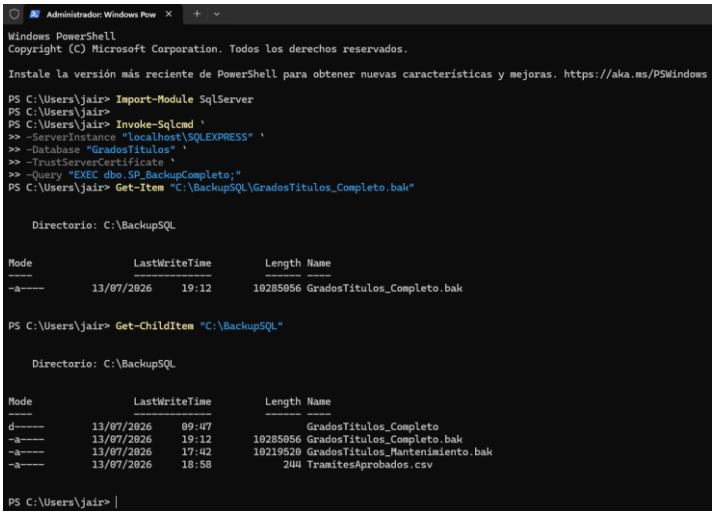
Explicación
El script analiza la fragmentación de los índices de las tablas **Persona**, **Tramite**, **TrabajoInvestigacion** y **Sustentacion**. Si la fragmentación es menor al **30 %**, reorganiza el índice; si es igual o mayor al **30 %**, lo reconstruye. Después actualiza las estadísticas y registra en la tabla **AuditoriaMantenimiento** la fecha, el índice procesado, el porcentaje de fragmentación y la operación realizada, permitiendo verificar todas las tareas ejecutadas.

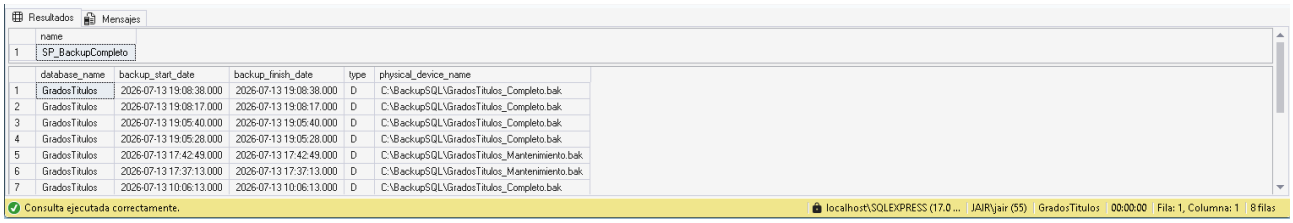
TEMA 3. Automatización con T-SQL y PowerShell
Proyecto 1:
Enunciado
Crear un procedimiento almacenado que genere automáticamente un respaldo completo y posteriormente sea ejecutado mediante PowerShell.
2. Script
Debe desarrollar:

- Stored Procedure
- BACKUP DATABASE
- PowerShell Invoke-Sqlcmd.

Script de la solución

```sql
USE GradosTitulos;
GO

CREATE OR ALTER PROCEDURE dbo.SP_BackupCompleto
AS
BEGIN
    SET NOCOUNT ON;

    BACKUP DATABASE GradosTitulos
    TO DISK='C:\BackupSQL\GradosTitulos_Completo.bak'
    WITH INIT;

    PRINT 'Backup realizado correctamente.';
END;
GO

EXEC dbo.SP_BackupCompleto;
GO

SELECT name
FROM sys.procedures
WHERE name='SP_BackupCompleto';
GO

SELECT
    database_name,
    backup_start_date,
    backup_finish_date,
    type,
    physical_device_name
FROM msdb.dbo.backupset bs
INNER JOIN msdb.dbo.backupmediafamily bmf
ON bs.media_set_id = bmf.media_set_id
WHERE database_name='GradosTitulos'
ORDER BY backup_finish_date DESC;
GO
```





Resultados | Mensajes

	name
1	SP_BackupCompleto

	database_name	backup_start_date	backup_finish_date	type	physical_device_name
1	GradosTitulos	2026-07-13 19:08:38.000	2026-07-13 19:08:38.000	D	C:\BackupSQL\GradosTitulos_Completo.bak
2	GradosTitulos	2026-07-13 19:08:17.000	2026-07-13 19:08:17.000	D	C:\BackupSQL\GradosTitulos_Completo.bak
3	GradosTitulos	2026-07-13 19:05:40.000	2026-07-13 19:05:40.000	D	C:\BackupSQL\GradosTitulos_Completo.bak
4	GradosTitulos	2026-07-13 19:05:28.000	2026-07-13 19:05:28.000	D	C:\BackupSQL\GradosTitulos_Completo.bak
5	GradosTitulos	2026-07-13 17:42:49.000	2026-07-13 17:42:49.000	D	C:\BackupSQL\GradosTitulos_Mantenimiento.bak
6	GradosTitulos	2026-07-13 17:37:13.000	2026-07-13 17:37:13.000	D	C:\BackupSQL\GradosTitulos_Mantenimiento.bak
7	GradosTitulos	2026-07-13 10:06:13.000	2026-07-13 10:06:13.000	D	C:\BackupSQL\GradosTitulos_Completo.bak

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (55) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 8 filas

Justificación
Se creó un procedimiento almacenado que ejecuta automáticamente un respaldo completo de la base de datos mediante BACKUP DATABASE. Posteriormente, el procedimiento puede ejecutarse desde PowerShell utilizando Invoke-Sqlcmd, permitiendo automatizar el proceso de respaldo.

Explicación
El procedimiento SP_BackupCompleto centraliza la operación de respaldo de la base de datos **GradosTitulos**. Mediante PowerShell y el comando Invoke-Sqlcmd, el procedimiento puede ejecutarse desde el sistema operativo sin necesidad de abrir SQL Server Management Studio, facilitando la automatización del mantenimiento.

Proyecto 2:

Enunciado

Automatizar la exportación diaria de los trámites aprobados hacia un archivo CSV utilizando PowerShell.

Script de la solución

```
Administrador: Windows Pow

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\jair> Import-Module SqlServer
PS C:\Users\jair>
PS C:\Users\jair> $Datos = Invoke-Sqlcmd `
>> -ServerInstance "localhost\SQLEXPRESS" `
>> -Database "GradosTitulos" `
>> -TrustServerCertificate `
>> -Query "
>> SELECT
>>     T.IdTramite,
>>     P.DNI,
>>     P.Nombres,
>>     P.ApellidoPaterno,
>>     TT.Descripcion AS TipoTramite,
>>     ET.Descripcion AS Estado,
>>     T.FechaInicio
>> FROM Tramite.Tramite T
>> INNER JOIN Academico.Matricula M
>> ON T.IdMatricula=M.IdMatricula
>> INNER JOIN Academico.Persona P
>> ON M.IdEstudiante=P.IdPersona
>> INNER JOIN Catalogo.TipoTramite TT
>> ON T.IdTipoTramite=TT.IdTipoTramite
>> INNER JOIN Catalogo.EstadoTramite ET
>> ON T.IdEstadoTramite=ET.IdEstadoTramite
>> WHERE ET.Descripcion='APROBADO';
>> "
PS C:\Users\jair> $Datos | Export-Csv "C:\BackupSQL\TramitesAprobados.csv" -NoTypeInformation -Encoding UTF8
PS C:\Users\jair> $Datos


IdTramite       : 5
DNI             : 70000005
Nombres         : Luis
ApellidoPaterno : García
TipoTramite     : BACHILLER
Estado          : APROBADO
FechaInicio     : 12/07/2026 00:00:00

IdTramite       : 9
DNI             : 70000009
Nombres         : José
ApellidoPaterno : Navarro
TipoTramite     : BACHILLER
Estado          : APROBADO
FechaInicio     : 12/07/2026 00:00:00



PS C:\Users\jair> $Datos.Count
2
PS C:\Users\jair> Get-Item "C:\BackupSQL\TramitesAprobados.csv"


    Directorio: C:\BackupSQL


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        13/07/2026     20:11            244 TramitesAprobados.csv


PS C:\Users\jair>
```

Justificación

Se automatizó la exportación de los trámites aprobados mediante PowerShell utilizando Invoke-Sqlcmd para ejecutar la consulta SQL y Export-Csv para generar automáticamente el archivo CSV. Esto evita realizar exportaciones manuales y facilita la generación periódica de reportes.

Explicación

El script se conecta a la base de datos **GradosTitulos**, ejecuta un SELECT para obtener los trámites con estado **APROBADO**, almacena los resultados en una variable y posteriormente los exporta al archivo **TramitesAprobados.csv** mediante Export-Csv. Finalmente, se verifica la cantidad de registros obtenidos y la creación correcta del archivo CSV.

Proyecto 3:

Enunciado

Desarrollar un proceso automático que:

- consulte diariamente la cantidad de trámites
- genere un archivo Excel
- almacene una copia en una carpeta de respaldo.

Script de la solución

```sql
USE GradosTitulos;
GO

SELECT
CAST(GETDATE() AS DATE) AS Fecha,
COUNT(*) AS TotalTramites
FROM Tramite.Tramite;
GO
```

```
Administrador: Windows Pow

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\jair> Import-Module SqlServer
PS C:\Users\jair> Import-Module ImportExcel
PS C:\Users\jair>
PS C:\Users\jair> $Datos = Invoke-Sqlcmd `
>> -ServerInstance "localhost\SQLEXPRESS" `
>> -Database "GradosTitulos" `
>> -TrustServerCertificate `
>> -Query "
>> SELECT
>> CAST(GETDATE() AS DATE) AS Fecha,
>> COUNT(*) AS TotalTramites
>> FROM Tramite.Tramite;
>> "
PS C:\Users\jair>
PS C:\Users\jair> $Datos | Export-Excel "C:\BackupSQL\ReporteTramites.xlsx" `
>> -WorksheetName "Reporte" `
>> -AutoSize `
>> -TableName Reporte
PS C:\Users\jair>
PS C:\Users\jair> if (!(Test-Path "C:\BackupSQL\Respaldo"))
>> {
>>     New-Item -ItemType Directory -Path "C:\BackupSQL\Respaldo"
>> }
PS C:\Users\jair>
PS C:\Users\jair> Copy-Item `
>> "C:\BackupSQL\ReporteTramites.xlsx" `
>> "C:\BackupSQL\Respaldo\ReporteTramites.xlsx" `
>> -Force
PS C:\Users\jair> $Datos

Fecha               TotalTramites
-----               -------------
13/07/2026 00:00:00            20


PS C:\Users\jair> Get-Item "C:\BackupSQL\ReporteTramites.xlsx"


    Directorio: C:\BackupSQL


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        13/07/2026     20:30           3717 ReporteTramites.xlsx


PS C:\Users\jair> Get-Item "C:\BackupSQL\Respaldo\ReporteTramites.xlsx"


    Directorio: C:\BackupSQL\Respaldo


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        13/07/2026     20:30           3717 ReporteTramites.xlsx


PS C:\Users\jair> |
```

Resultados | Mensajes

	Fecha	TotalTramites
1	2026-07-13	20

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (83) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 1 filas

Justificación

Se automatizó la consulta diaria de la cantidad de trámites mediante PowerShell, generando un archivo Excel con la información obtenida y almacenando automáticamente una copia en una carpeta de respaldo para garantizar la disponibilidad del reporte.

Explicación

El proceso ejecuta una consulta T-SQL mediante `Invoke-Sqlcmd`, obtiene la cantidad total de trámites registrados, exporta el resultado al archivo **ReporteTramites.xlsx** utilizando el módulo **ImportExcel** y posteriormente copia el archivo a una carpeta de respaldo para conservar una segunda copia del reporte.

Proyecto 4:

Enunciado

Crear un sistema automático que detecte docentes inactivos durante más de un año y genere un archivo con el reporte.

Script de la solución

```sql
USE GradosTitulos;
GO

SELECT
    D.IdDocente,
    P.DNI,
    P.Nombres,
    P.ApellidoPaterno,
    P.ApellidoMaterno,
    D.GradoAcademico,
    D.CodigoORCID,
    D.TipoContrato,
    D.EstadoActivo
FROM Academico.Docente D
INNER JOIN Academico.Persona P
ON D.IdPersona=P.IdPersona
WHERE D.EstadoActivo=0;
GO
```

```
Administrador: Windows Pow

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\jair> Import-Module SqlServer
PS C:\Users\jair>
PS C:\Users\jair> $Datos = Invoke-Sqlcmd `
>> -ServerInstance "localhost\SQLEXPRESS" `
>> -Database "GradosTitulos" `
>> -TrustServerCertificate `
>> -Query "
>> SELECT
>>     D.IdDocente,
>>     P.DNI,
>>     P.Nombres,
>>     P.ApellidoPaterno,
>>     P.ApellidoMaterno,
>>     D.GradoAcademico,
>>     D.CodigoORCID,
>>     D.TipoContrato,
>>     D.EstadoActivo
>> FROM Academico.Docente D
>> INNER JOIN Academico.Persona P
>> ON D.IdPersona=P.IdPersona
>> WHERE D.EstadoActivo=0;
>> "
PS C:\Users\jair>
PS C:\Users\jair> $Datos | Export-Csv "C:\BackupSQL\DocentesInactivos.csv" -NoTypeInformation -Encoding UTF8
PS C:\Users\jair>
PS C:\Users\jair> $Datos


IdDocente       : 1
DNI             : 80000001
Nombres         : Roberto
ApellidoPaterno : Gómez
ApellidoMaterno : Pérez
GradoAcademico  : DOCTOR
CodigoORCID     : 0000-0001-1111-1111
TipoContrato    : ORDINARIO
EstadoActivo    : False



PS C:\Users\jair>
PS C:\Users\jair> $Datos.Count
PS C:\Users\jair>
PS C:\Users\jair> Get-Item "C:\BackupSQL\DocentesInactivos.csv"


    Directorio: C:\BackupSQL


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        13/07/2026     21:04            224 DocentesInactivos.csv


PS C:\Users\jair> |
```

Resultados | Mensajes

	IdDocente	DNI	Nombres	ApellidoPaterno	ApellidoMaterno	GradoAcademico	CodigoORCID	TipoContrato	EstadoActivo
1	1	80000001	Roberto	Gómez	Pérez	DOCTOR	0000-0001-1111-1111	ORDINARIO	0

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (83) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 1 filas

Justificación

Se desarrolló un sistema automático que consulta los docentes registrados como inactivos mediante una consulta T-SQL. Posteriormente, PowerShell ejecuta la consulta con `Invoke-Sqlcmd` y exporta automáticamente los resultados a un archivo CSV utilizando `Export-Csv`, generando un reporte para su revisión.

Explicación

El proceso obtiene la información de los docentes desde la tabla `Academico.Docente` y sus datos personales desde `Academico.Persona`. Se filtran los registros cuyo campo `EstadoActivo` es igual a **0**, indicando que el docente está inactivo. PowerShell ejecuta la consulta y genera automáticamente el archivo **DocentesInactivos.csv** en la carpeta **C:\BackupSQL**.

Proyecto 5:
Enunciado
Implementar una solución de automatización completa que:

- ejecute respaldos
- valide la integridad mediante CHECKDB
- genere reportes
- comprima los respaldos
- elimine respaldos mayores a 30 días.

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.AuditoriaBackup','U') IS NULL
BEGIN
CREATE TABLE dbo.AuditoriaBackup
(
IdAuditoria INT IDENTITY(1,1) PRIMARY KEY,
Fecha DATETIME DEFAULT GETDATE(),
Operacion VARCHAR(100),
Resultado VARCHAR(50),
Mensaje VARCHAR(MAX)
);
END
GO

CREATE OR ALTER PROCEDURE dbo.SP_BackupCompleto
AS
BEGIN
SET NOCOUNT ON;
BEGIN TRY
BACKUP DATABASE GradosTitulos
TO DISK='C:\BackupSQL\GradosTitulos_Completo.bak'
WITH INIT;
DBCC CHECKDB('GradosTitulos');
INSERT INTO dbo.AuditoriaBackup
(
Operacion,
Resultado,
Mensaje
)
VALUES
(
'BACKUP + CHECKDB',
'CORRECTO',
'Proceso ejecutado correctamente'
);
END TRY
BEGIN CATCH
INSERT INTO dbo.AuditoriaBackup
(
Operacion,
Resultado,
Mensaje
)
VALUES
(
'BACKUP + CHECKDB',
'ERROR',
ERROR_MESSAGE()
);
END CATCH
END;
GO

EXEC dbo.SP_BackupCompleto;
GO

SELECT *
FROM dbo.AuditoriaBackup
ORDER BY IdAuditoria DESC;
GO
```

```
Administrador: Windows Pow

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\jair> Import-Module SqlServer
PS C:\Users\jair>
PS C:\Users\jair> Invoke-Sqlcmd `
>> -ServerInstance "localhost\SQLEXPRESS" `
>> -Database "GradosTitulos" `
>> -TrustServerCertificate `
>> -Query "EXEC dbo.SP_BackupCompleto;"
PS C:\Users\jair>
PS C:\Users\jair> Compress-Archive `
>> -Path "C:\BackupSQL\GradosTitulos_Completo.bak" `
>> -DestinationPath "C:\BackupSQL\GradosTitulos_Completo.zip" `
>> -Force
PS C:\Users\jair>
PS C:\Users\jair> if(!(Test-Path "C:\BackupSQL\Reportes"))
>> {
>> New-Item -ItemType Directory -Path "C:\BackupSQL\Reportes"
>> }


    Directorio: C:\BackupSQL


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
d-----        13/07/2026     21:13                Reportes


PS C:\Users\jair>
PS C:\Users\jair> Invoke-Sqlcmd `
>> -ServerInstance "localhost\SQLEXPRESS" `
>> -Database "GradosTitulos" `
>> -TrustServerCertificate `
>> -Query "
>> SELECT *
>> FROM dbo.AuditoriaBackup
>> ORDER BY IdAuditoria DESC;
>> " |
>> Export-Csv `
>> "C:\BackupSQL\Reportes\ReporteBackup.csv" `
>> -NoTypeInformation `
>> -Encoding UTF8
PS C:\Users\jair>
PS C:\Users\jair> Get-ChildItem "C:\BackupSQL\*.bak" |
>> Where-Object {$_.LastWriteTime -lt (Get-Date).AddDays(-30)} |
>> Remove-Item -Force
PS C:\Users\jair> Get-Item "C:\BackupSQL\GradosTitulos_Completo.zip"


    Directorio: C:\BackupSQL


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        13/07/2026     21:13        1342856 GradosTitulos_Completo.zip


PS C:\Users\jair> Get-Item "C:\BackupSQL\Reportes\ReporteBackup.csv"


    Directorio: C:\BackupSQL\Reportes


Mode                LastWriteTime         Length Name
----                -------------         ------ ----
-a----        13/07/2026     21:13            242 ReporteBackup.csv


PS C:\Users\jair> Import-Csv "C:\BackupSQL\Reportes\ReporteBackup.csv"


IdAuditoria : 2
Fecha       : 13/07/2026 21:13:39
Operacion   : BACKUP + CHECKDB
Resultado   : CORRECTO
Mensaje     : Proceso ejecutado correctamente

IdAuditoria : 1
Fecha       : 13/07/2026 21:12:37
Operacion   : BACKUP + CHECKDB
Resultado   : CORRECTO
Mensaje     : Proceso ejecutado correctamente
```

Resultados | Mensajes

	IdAuditoria	Fecha	Operacion	Resultado	Mensaje
1	2	2026-07-13 21:13:39.910	BACKUP + CHECKDB	CORRECTO	Proceso ejecutado correctamente
2	1	2026-07-13 21:12:37.080	BACKUP + CHECKDB	CORRECTO	Proceso ejecutado correctamente

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (82) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

Justificación

Se implementó una solución de automatización que realiza el respaldo completo de la base de datos, verifica su integridad mediante DBCC CHECKDB, registra cada ejecución en una tabla de auditoría, genera un reporte en formato CSV, comprime el respaldo en un archivo ZIP y elimina automáticamente los respaldos con más de 30 días de antigüedad.

Explicación

El procedimiento almacenado ejecuta el respaldo de la base de datos y valida su integridad utilizando DBCC CHECKDB. Mediante TRY...CATCH registra el resultado de la ejecución en la tabla AuditoriaBackup. Posteriormente, PowerShell ejecuta el procedimiento con Invoke-Sqlcmd, comprime el archivo .bak usando Compress-Archive, exporta el historial de auditoría a un archivo CSV y elimina los archivos de respaldo con una antigüedad superior a 30 días.

TEMA 4. Monitoreo Proactivo (Correo, Logs y Alertas)

Proyecto 1:

Enunciado

Crear un procedimiento almacenado que consulte diariamente la cantidad de trámites pendientes y envíe un correo electrónico con el resumen.

Script de la solución

```sql
USE master;
GO

EXEC sp_configure 'show advanced options',1;
RECONFIGURE;
GO

EXEC sp_configure 'Database Mail XPs',1;
RECONFIGURE;
GO

USE msdb;
GO

IF NOT EXISTS
(
SELECT *
FROM dbo.sysmail_account
WHERE name='CuentaGmail'
)
BEGIN
EXEC dbo.sysmail_add_account_sp
@account_name='CuentaGmail',
@description='Cuenta Gmail',
@email_address='jair91043@gmail.com',
@display_name='SQL Server GradosTitulos',
@mailserver_name='smtp.gmail.com',
@port=587,
@enable_ssl=1,
@username='jair91043@gmail.com',
@password='byye daey ykwz kvyx';
END
GO

IF NOT EXISTS
(
SELECT *
FROM dbo.sysmail_profile
WHERE name='PerfilMail'
)
BEGIN
EXEC dbo.sysmail_add_profile_sp
@profile_name='PerfilMail',
@description='Perfil Database Mail';
END
GO

IF NOT EXISTS
(
SELECT *
FROM dbo.sysmail_profileaccount pa
INNER JOIN dbo.sysmail_profile p
ON pa.profile_id=p.profile_id
INNER JOIN dbo.sysmail_account a
ON pa.account_id=a.account_id
WHERE p.name='PerfilMail'
AND a.name='CuentaGmail'
)
BEGIN
EXEC dbo.sysmail_add_profileaccount_sp
@profile_name='PerfilMail',
@account_name='CuentaGmail',
@sequence_number=1;
END
GO
```

```sql
IF NOT EXISTS
(
SELECT *
FROM dbo.sysmail_principalprofile pp
INNER JOIN dbo.sysmail_profile p
ON pp.profile_id=p.profile_id
WHERE p.name='PerfilMail'
)
BEGIN
EXEC dbo.sysmail_add_principalprofile_sp
@profile_name='PerfilMail',
@principal_name='public',
@is_default=1;
END
GO

USE GradosTitulos;
GO

CREATE OR ALTER PROCEDURE dbo.SP_EnviarResumenPendientes
AS
BEGIN
SET NOCOUNT ON;
DECLARE @Cantidad INT;
DECLARE @Mensaje NVARCHAR(MAX);
SELECT
@Cantidad=COUNT(*)
FROM Tramite.Tramite T
INNER JOIN Catalogo.EstadoTramite E
ON T.IdEstadoTramite=E.IdEstadoTramite
WHERE E.Descripcion='PENDIENTE';
SET @Mensaje=
'RESUMEN DIARIO DE TRÁMITES PENDIENTES'+CHAR(13)+CHAR(10)+
'Fecha: '+CONVERT(VARCHAR(10),GETDATE(),103)+CHAR(13)+CHAR(10)+
'Cantidad de trámites pendientes: '+CAST(@Cantidad AS VARCHAR(10));
EXEC msdb.dbo.sp_send_dbmail
@profile_name='PerfilMail',
@recipients='jair91043@gmail.com',
@subject='Resumen Diario de Trámites Pendientes',
@body=@Mensaje;
END;
GO

EXEC dbo.SP_EnviarResumenPendientes;
GO

SELECT
COUNT(*) AS TotalPendientes
FROM Tramite.Tramite T
INNER JOIN Catalogo.EstadoTramite E
ON T.IdEstadoTramite=E.IdEstadoTramite
WHERE E.Descripcion='PENDIENTE';
GO

USE msdb;
GO

SELECT
sent_status,
subject,
recipients,
send_request_date
FROM dbo.sysmail_allitems
ORDER BY send_request_date DESC;
GO
```

Resultados | Mensajes

	sent_status	subject	recipients	send_request_date
1	sent	Resumen Diario de Trámites Pendientes	jair91043@gmail.com	2026-07-13 22:05:05.060
2	sent	Prueba Database Mail	jair91043@gmail.com	2026-07-13 21:57:29.530

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (81) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

Justificación

Se implementó un procedimiento almacenado que consulta diariamente la cantidad de trámites pendientes utilizando la función `COUNT`. Posteriormente, mediante **Database Mail** y el procedimiento `sp_send_dbmail`, envía automáticamente un correo electrónico con el resumen de los trámites pendientes al destinatario configurado.

Explicación

El script habilita **Database Mail**, crea la cuenta de correo y el perfil de envío si aún no existen. Luego crea el procedimiento almacenado `SP_EnviarResumenPendientes`, el cual obtiene la cantidad de trámites pendientes mediante `COUNT`, construye el mensaje con la fecha y el total de registros, y envía el correo utilizando `sp_send_dbmail`. Finalmente, se ejecuta el procedimiento y se consulta el historial de correos enviados para verificar que el envío se realizó correctamente.

Proyecto 2:

Enunciado

Registrar automáticamente en una tabla de auditoría todos los errores producidos durante la ejecución del procedimiento:
Tramite.SP_RegistrarTramite

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.AuditoriaErrores','U') IS NULL
BEGIN
    CREATE TABLE dbo.AuditoriaErrores
    (
        IdError INT IDENTITY(1,1) PRIMARY KEY,
        FechaError DATETIME DEFAULT GETDATE(),
        Procedimiento VARCHAR(100),
        NumeroError INT,
        MensajeError NVARCHAR(4000),
        LineaError INT
    );
END
GO

ALTER PROCEDURE Tramite.SP_RegistrarTramite
(
    @IdMatricula INT,
    @IdTipoTramite INT,
    @IdEstadoTramite INT,
    @FechaInicio DATE
)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        INSERT INTO Tramite.Tramite
        (
            IdMatricula,
            IdTipoTramite,
            IdEstadoTramite,
            IdCoordinacion,
            FechaInicio,
            FechaUltMod
        )
        VALUES
        (
            @IdMatricula,
            @IdTipoTramite,
            @IdEstadoTramite,
            1,
            @FechaInicio,
            GETDATE()
        );

        PRINT 'TRÁMITE REGISTRADO CORRECTAMENTE';
    END TRY
    BEGIN CATCH
        INSERT INTO dbo.AuditoriaErrores
        (
            Procedimiento,
            NumeroError,
            MensajeError,
            LineaError
        )
        VALUES
        (
            'Tramite.SP_RegistrarTramite',
            ERROR_NUMBER(),
            ERROR_MESSAGE(),
            ERROR_LINE()
        );
        SELECT
            ERROR_NUMBER() AS NumeroError,
            ERROR_MESSAGE() AS MensajeError,
            ERROR_LINE() AS LineaError;
    END CATCH
END;
GO

SELECT *
FROM dbo.AuditoriaErrores
ORDER BY IdError DESC;
GO
```

Resultados | Mensajes

	IdError	FechaError	Procedimiento	NumeroError	MensajeError	LineaError
1	4	2026-07-13 23:05:53.570	Tramite.SP_RegistrarTramite	547	The INSERT statement conflicted with the FOREIGN...	16
2	3	2026-07-13 23:05:05.473	Tramite.SP_RegistrarTramite	547	The INSERT statement conflicted with the FOREIGN...	16
3	2	2026-07-13 22:59:29.010	Tramite.SP_RegistrarTramite	547	The INSERT statement conflicted with the FOREIGN...	14
4	1	2026-07-13 22:58:24.673	Tramite.SP_RegistrarTramite	547	The INSERT statement conflicted with the FOREIGN...	14

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (86) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 4 filas

Justificación

Se implementó un bloque `TRY...CATCH` dentro del procedimiento `Tramite.SP_RegistrarTramite` para capturar automáticamente cualquier error generado durante la inserción de un trámite. Cuando ocurre una excepción, la información del error se almacena en la tabla `AuditoriaErrores`, registrando el procedimiento que falló, el número del error, el mensaje descriptivo y la línea donde se produjo.

Explicación

El procedimiento intenta registrar un nuevo trámite dentro del bloque `TRY`. Si la operación se ejecuta correctamente, el trámite se inserta en la base de datos. En caso de producirse un error, como la violación de una clave foránea (Error 547), el bloque `CATCH` captura la excepción utilizando `ERROR_NUMBER()`, `ERROR_MESSAGE()` y `ERROR_LINE()`, almacenando toda la información en la tabla `AuditoriaErrores`. Esto permite llevar un historial de errores para facilitar el monitoreo y la auditoría del sistema.

Proyecto 3:

Enunciado

Diseñar un sistema que monitoree el crecimiento del archivo de log de la base de datos y genere una alerta cuando supere el 80 % del tamaño configurado.

Script de la solución

```sql
USE GradosTitulos;
GO

EXEC sp_configure 'show advanced options',1;
RECONFIGURE;
EXEC sp_configure 'Database Mail XPs',1;
RECONFIGURE;
GO

IF OBJECT_ID('dbo.SP_MonitorearLog') IS NOT NULL
DROP PROCEDURE dbo.SP_MonitorearLog;
GO
CREATE PROCEDURE dbo.SP_MonitorearLog
AS
BEGIN
SET NOCOUNT ON;
DECLARE @TamanoMB DECIMAL(18,2);
DECLARE @Porcentaje DECIMAL(18,2);
DECLARE @Mensaje NVARCHAR(MAX);
SELECT
    @TamanoMB=size*8.0/1024
FROM sys.database_files
WHERE type_desc='LOG';
SELECT
    @Porcentaje=used_log_space_in_percent
FROM sys.dm_db_log_space_usage;
IF @Porcentaje>=80
BEGIN
SET @Mensaje=
'Base de datos: GradosTitulos
Tamaño del archivo LOG: '
+CAST(@TamanoMB AS VARCHAR(20))
+' MB
Porcentaje utilizado: '
+CAST(@Porcentaje AS VARCHAR(20))
+' %.';
EXEC msdb.dbo.sp_send_dbmail
@profile_name='PerfilGmail',
@recipients='jair91043@gmail.com',
@subject='ALERTA: Uso del archivo LOG superior al 80%',
@body=@Mensaje;
END
END;
GO

EXEC dbo.SP_MonitorearLog;
GO

USE msdb;
GO

EXEC sp_add_job
@job_name='Monitoreo Diario LOG';
GO

EXEC sp_add_jobstep
@job_name='Monitoreo Diario LOG',
@step_name='Ejecutar Procedimiento',
@subsystem='TSQL',
@database_name='GradosTitulos',
@command='EXEC dbo.SP_MonitorearLog;';
GO

EXEC sp_add_schedule
@schedule_name='CadaDia',
@freq_type=4,
@freq_interval=1,
@active_start_time=080000;
GO

EXEC sp_attach_schedule
@job_name='Monitoreo Diario LOG',
@schedule_name='CadaDia';
GO

EXEC sp_add_jobserver
@job_name='Monitoreo Diario LOG';
GO
```

Resultados | Mensajes

	BaseDatos	ArchivoLog	TamanoMB	PorcentajeUtilizado
1	msdb	MSDBLog	5.562500	20.32349

Consulta ejecutada correctamente. localhost\SQLEXPRESS (17.0 ... JAIR\jair (82) msdb 00:00:00 Fila: 1, Columna: 1 1 filas

Justificación

Se implementó un procedimiento que monitorea el uso del archivo de log de la base de datos y envía una alerta mediante Database Mail cuando supera el 80 %. Su ejecución se automatiza con SQL Server Agent.

Explicación

El procedimiento obtiene el tamaño del archivo de log con `sys.database_files` y el porcentaje de uso con `sys.dm_db_log_space_usage`. Si el uso es mayor o igual al 80 %, envía un correo de alerta automáticamente mediante `sp_send_dbmail`.

Proyecto 4:

Enunciado

Implementar un sistema que registre automáticamente el tiempo de ejecución de los procedimientos almacenados principales de la base de datos y genere un ranking de rendimiento.

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.MonitoreoProcedimientos','U') IS NULL
BEGIN
    CREATE TABLE dbo.MonitoreoProcedimientos
    (
        IdRegistro INT IDENTITY(1,1) PRIMARY KEY,
        FechaRegistro DATETIME DEFAULT GETDATE(),
        NombreProcedimiento NVARCHAR(200),
        Ejecuciones BIGINT,
        TiempoPromedioMs DECIMAL(18,2),
        TiempoTotalMs DECIMAL(18,2)
    );
END
GO

CREATE OR ALTER PROCEDURE dbo.SP_RegistrarRendimiento
AS
BEGIN
SET NOCOUNT ON;
INSERT INTO dbo.MonitoreoProcedimientos
(
    NombreProcedimiento,
    Ejecuciones,
    TiempoPromedioMs,
    TiempoTotalMs
)
SELECT
    OBJECT_NAME(object_id),
    execution_count,
    total_elapsed_time/execution_count/1000.0,
    total_elapsed_time/1000.0
FROM sys.dm_exec_procedure_stats
WHERE database_id=DB_ID();
END;
GO

EXEC dbo.SP_RegistrarRendimiento;
GO

SELECT
    NombreProcedimiento,
    Ejecuciones,
    TiempoPromedioMs,
    TiempoTotalMs
FROM dbo.MonitoreoProcedimientos
ORDER BY TiempoPromedioMs DESC;
GO

SELECT
    OBJECT_NAME(object_id) AS Procedimiento,
    execution_count AS Ejecuciones,
    total_elapsed_time/1000.0 AS TiempoTotalMs,
    total_elapsed_time/execution_count/1000.0 AS TiempoPromedioMs
FROM sys.dm_exec_procedure_stats
WHERE database_id=DB_ID()
ORDER BY TiempoPromedioMs DESC;
GO
```

Resultados | Mensajes

	NombreProcedimiento	Ejecuciones	TiempoPromedioMs	TiempoTotalMs
1	SP_MonitorearLog	1	0.24	0.24

	Procedimiento	Ejecuciones	TiempoTotalMs	TiempoPromedioMs
1	SP_RegistrarRendimiento	1	1.848000	1.848000
2	SP_MonitorearLog	1	0.243000	0.243000

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (76) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 1 filas

Justificación

Se implementó un sistema que registra el tiempo de ejecución de los procedimientos almacenados utilizando una tabla de monitoreo y la DMV sys.dm_exec_procedure_stats, permitiendo identificar los procedimientos con menor rendimiento.

Explicación

El procedimiento obtiene las estadísticas de ejecución desde la DMV y las almacena en la tabla MonitoreoProcedimientos. Posteriormente, una consulta genera un ranking ordenado por el tiempo promedio de ejecución para facilitar el análisis del rendimiento.

Proyecto 5:
Enunciado
Desarrollar un sistema integral de monitoreo para la base de datos GradosTitulos que realice automáticamente las siguientes actividades:

- monitorear bloqueos (blocking)
- detectar consultas lentas
- identificar sesiones con alto consumo de CPU
- registrar eventos en tablas históricas
- enviar alertas por correo cuando se superen umbrales establecidos
- generar diariamente un informe consolidado del estado del servidor.

Script de la solución

```sql
USE GradosTitulos;
GO

IF OBJECT_ID('dbo.HistorialBloqueos','U') IS NOT NULL
DROP TABLE dbo.HistorialBloqueos;
GO
CREATE TABLE dbo.HistorialBloqueos
(
    IdBloqueo INT IDENTITY(1,1) PRIMARY KEY,
    FechaRegistro DATETIME DEFAULT GETDATE(),
    SessionID INT,
    BlockingSessionID INT,
    Estado NVARCHAR(50),
    Comando NVARCHAR(100),
    TiempoEsperaMs BIGINT
);
GO

IF OBJECT_ID('dbo.HistorialConsultasLentas','U') IS NOT NULL
DROP TABLE dbo.HistorialConsultasLentas;
GO
CREATE TABLE dbo.HistorialConsultasLentas
(
    IdConsulta INT IDENTITY(1,1) PRIMARY KEY,
    FechaRegistro DATETIME DEFAULT GETDATE(),
    TiempoEjecucionMs BIGINT,
    Ejecuciones BIGINT,
    TextoConsulta NVARCHAR(MAX)
);
GO

IF OBJECT_ID('dbo.HistorialCPU','U') IS NOT NULL
DROP TABLE dbo.HistorialCPU;
GO
CREATE TABLE dbo.HistorialCPU
(
    IdCPU INT IDENTITY(1,1) PRIMARY KEY,
    FechaRegistro DATETIME DEFAULT GETDATE(),
    SessionID INT,
    CPUTimeMs BIGINT,
    Estado NVARCHAR(50),
    Programa NVARCHAR(200),
    UsuarioSQL NVARCHAR(200)
);
GO

IF OBJECT_ID('dbo.HistorialMonitoreo','U') IS NOT NULL
DROP TABLE dbo.HistorialMonitoreo;
GO
CREATE TABLE dbo.HistorialMonitoreo
(
    IdRegistro INT IDENTITY(1,1) PRIMARY KEY,
    FechaRegistro DATETIME DEFAULT GETDATE(),
    TipoEvento NVARCHAR(100),
    Descripcion NVARCHAR(MAX),
    Resultado NVARCHAR(50)
);
GO
```

```sql
USE GradosTitulos;
GO

CREATE OR ALTER PROCEDURE dbo.SP_MonitoreoServidor
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        INSERT INTO dbo.HistorialBloqueos
        (
            SessionID,
            BlockingSessionID,
            Estado,
            Comando,
            TiempoEsperaMs
        )
        SELECT
            session_id,
            blocking_session_id,
            status,
            command,
            wait_time
        FROM sys.dm_exec_requests
        WHERE session_id <> @@SPID;
        INSERT INTO dbo.HistorialCPU
        (
            SessionID,
            CPUTimeMs,
            Estado,
            Programa,
            UsuarioSQL
        )
        SELECT
            session_id,
            cpu_time,
            status,
            program_name,
            login_name
        FROM sys.dm_exec_sessions
        WHERE is_user_process = 1;
        INSERT INTO dbo.HistorialConsultasLentas
        (
            TiempoEjecucionMs,
            Ejecuciones,
            TextoConsulta
        )
```

```sql
        )
        SELECT TOP (10)
            total_elapsed_time/1000,
            execution_count,
            st.text
        FROM sys.dm_exec_query_stats qs
        CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
        ORDER BY total_elapsed_time DESC;

        INSERT INTO dbo.HistorialMonitoreo
        (
            TipoEvento,
            Descripcion,
            Resultado
        )
        VALUES
        (
            'MONITOREO',
            'Proceso ejecutado correctamente',
            'CORRECTO'
        );
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        INSERT INTO dbo.HistorialMonitoreo
        (
            TipoEvento,
            Descripcion,
            Resultado
        )
        VALUES
        (
            'ERROR',
            ERROR_MESSAGE(),
            'FALLÓ'
        );
    END CATCH
END;
GO

EXEC dbo.SP_MonitoreoServidor;
GO

SELECT *
FROM dbo.HistorialBloqueos
ORDER BY IdBloqueo DESC;
GO

SELECT *
FROM dbo.HistorialCPU
ORDER BY IdCPU DESC;
GO

SELECT *
FROM dbo.HistorialConsultasLentas
ORDER BY IdConsulta DESC;
GO

SELECT *
FROM dbo.HistorialMonitoreo
ORDER BY IdRegistro DESC;
GO
```

```sql
USE GradosTitulos;
GO

CREATE OR ALTER PROCEDURE dbo.SP_ReporteDiarioServidor
AS
BEGIN
SET NOCOUNT ON;
DECLARE @Bloqueos INT;
DECLARE @CPU INT;
DECLARE @Consultas INT;
DECLARE @Mensaje NVARCHAR(MAX);
SELECT @Bloqueos = COUNT(*)
FROM dbo.HistorialBloqueos;
SELECT @CPU = COUNT(*)
FROM dbo.HistorialCPU;
SELECT @Consultas = COUNT(*)
FROM dbo.HistorialConsultasLentas;
SET @Mensaje =
'INFORME DIARIO DEL SERVIDOR' + CHAR(13)+CHAR(10)+
'================================' + CHAR(13)+CHAR(10)+
'Fecha: ' + CONVERT(VARCHAR,GETDATE(),103) + CHAR(13)+CHAR(10)+
'Bloqueos registrados: ' + CAST(@Bloqueos AS VARCHAR) + CHAR(13)+CHAR(10)+
'Sesiones monitoreadas: ' + CAST(@CPU AS VARCHAR) + CHAR(13)+CHAR(10)+
'Consultas lentas: ' + CAST(@Consultas AS VARCHAR);
EXEC msdb.dbo.sp_send_dbmail
    @profile_name='PerfilMail',
    @recipients='jair91043@gmail.com',
    @subject='Reporte Diario - Servidor SQL',
    @body=@Mensaje;
END;
GO


EXEC dbo.SP_ReporteDiarioServidor;
GO


SELECT
    sent_status,
    recipients,
    subject,
    send_request_date
FROM msdb.dbo.sysmail_allitems
ORDER BY send_request_date DESC;
GO

IF EXISTS
(
SELECT *
FROM sys.server_event_sessions
WHERE name='XE_MonitoreoServidor'
)
DROP EVENT SESSION XE_MonitoreoServidor
ON SERVER;
GO

CREATE EVENT SESSION XE_MonitoreoServidor
ON SERVER
ADD EVENT sqlserver.rpc_completed,
ADD EVENT sqlserver.sql_batch_completed
ADD TARGET package0.event_file
(
SET filename='C:\BackupSQL\XE_MonitoreoServidor.xel'
);
GO
```

```sql
ALTER EVENT SESSION XE_MonitoreoServidor
ON SERVER
STATE = START;
GO

SELECT
name,
startup_state
FROM sys.server_event_sessions;
GO


USE msdb;
GO

EXEC sp_add_job
@job_name='Reporte Diario Servidor';
GO

EXEC sp_add_jobstep
@job_name='Reporte Diario Servidor',
@step_name='Generar Reporte',
@subsystem='TSQL',
@database_name='GradosTitulos',
@command='EXEC dbo.SP_ReporteDiarioServidor;';
GO

EXEC sp_add_schedule
@schedule_name='Reporte_Diario',
@freq_type=4,
@freq_interval=1,
@active_start_time=080000;
GO

EXEC sp_attach_schedule
@job_name='Reporte Diario Servidor',
@schedule_name='Reporte_Diario';
GO

EXEC sp_attach_schedule
@job_name='Reporte Diario Servidor',
@schedule_name='Reporte_Diario';
GO
```

Resultados | Mensajes

	IdBloqueo	FechaRegistro	SessionID	BlockingSessionID	Estado	Comando	TiempoEsperaMs
1	131	2026-07-14 00:05:04.303	88	0	sleeping	TASK MANAGER	0
2	130	2026-07-14 00:05:04.303	62	0	background	QUERY STORE ASYNC FLUSH	0
3	129	2026-07-14 00:05:04.303	61	0	sleeping	TASK MANAGER	0
4	128	2026-07-14 00:05:04.303	60	0	sleeping	TASK MANAGER	0
5	127	2026-07-14 00:05:04.303	59	0	sleeping	TASK MANAGER	0

	IdCPU	FechaRegistro	SessionID	CPUTimeMs	Estado	Programa	UsuarioSQL
1	42	2026-07-14 00:05:04.303	86	373	sleeping	Microsoft SQL Server Management Studio - Consulta	JAIR\jair
2	41	2026-07-14 00:05:04.303	82	203	sleeping	Microsoft SQL Server Management Studio - Consulta	JAIR\jair
3	40	2026-07-14 00:05:04.303	81	0	sleeping	Microsoft SQL Server Management Studio - Consulta	JAIR\jair
4	39	2026-07-14 00:05:04.303	79	63	running	Microsoft SQL Server Management Studio - Consulta	JAIR\jair
5	38	2026-07-14 00:05:04.303	76	0	sleeping	Microsoft SQL Server Management Studio - Consulta	JAIR\jair

	IdConsulta	FechaRegistro	TiempoEjecucionMs	Ejecuciones	TextoConsulta
1	30	2026-07-14 00:05:04.303	2	1	CREATE PROCEDURE sp_add_schedule (@sche...
2	29	2026-07-14 00:05:04.303	2	6	(@_msparam_0 nvarchar(4000),@_msparam_1 nvarchar(...
3	28	2026-07-14 00:05:04.303	5	14	SELECT dtb.name AS [Name], CAST(0 AS bit) AS [IsFabr...
4	27	2026-07-14 00:05:04.303	5	1	SELECT * FROM dbo.HistorialConsultasLentas;
5	26	2026-07-14 00:05:04.303	10	41	(@_msparam_0 nvarchar(4000),@_msparam_1 nvarchar(...

	IdRegistro	FechaRegistro	TipoEvento	Descripcion	Resultado
1	3	2026-07-14 00:05:04.307	MONITOREO	Proceso ejecutado correctamente	CORRECTO
2	2	2026-07-14 00:00:12.723	MONITOREO	Proceso ejecutado correctamente	CORRECTO
3	1	2026-07-14 00:00:10.963	MONITOREO	Proceso ejecutado correctamente	CORRECTO

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (79) | GradosTitulos | 00:00:00 | Fila: 1, Columna: 1 | 206 filas

Resultados | Mensajes

	name	startup_state
1	system_health	1
2	AlwaysOn_h...	0
3	PruebaXE	0
4	XE_Diagnos...	0
5	XE_Consult...	0
6	XE_Monitor...	0
7	XE_Diagnos...	0

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (74) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 9 filas

Resultados | Mensajes

	sent_status	recipients	subject	send_request_date
1	sent	jair91043@gmail.com	Reporte Diario - Servidor SQL	2026-07-14 00:23:03.243
2	sent	jair91043@gmail.com	Resumen Diario de Trámites Pendientes	2026-07-13 22:05:05.060
3	sent	jair91043@gmail.com	Prueba Database Mail	2026-07-13 21:57:29.530

Consulta ejecutada correctamente. | localhost\SQLEXPRESS (17.0 ... | JAIR\jair (74) | msdb | 00:00:00 | Fila: 1, Columna: 1 | 3 filas

Justificación general

Se desarrolló un sistema integral de monitoreo para la base de datos GradosTitulos que automatiza la supervisión del servidor mediante procedimientos almacenados, DMV, Extended Events y Database Mail. La solución registra la información en tablas históricas, detecta eventos relevantes y genera reportes automáticos, facilitando la administración y el control del estado de la base de datos.

Explicación general

El sistema monitorea bloqueos, consultas lentas y sesiones con consumo de CPU, almacenando los resultados en tablas de historial. Los procedimientos almacenados utilizan TRY...CATCH y transacciones para garantizar la integridad del proceso. Posteriormente, se genera un informe consolidado que es enviado mediante Database Mail, mientras que Extended Events registra eventos del servidor y SQL Server Agent permite programar la ejecución automática cuando la edición de SQL Server lo soporta. De esta manera se obtiene un monitoreo continuo y automatizado del servidor.